

DOCUMENTACIÓN TÉCNICA DEL SISTEMA — RENTALBOOSTED

1. Introducción

RentalBoosted es el sistema ERP interno de Casa Drywall E.I.R.L., construido en Laravel 12 (PHP 8.2+, MySQL, Blade, Vite), que administra ventas (con y sin comprobante electrónico), compras, inventario multi-almacén, punto de venta, finanzas y recursos humanos, y da soporte a una segunda empresa (Jitk) sobre la misma instalación con bases de datos separadas.

Este documento usa como marco de referencia la norma ISO/IEC 25010 de calidad de producto software para estructurar la evaluación técnica, y sigue un principio único: toda afirmación se respalda con evidencia verificable en el código, la configuración, la base de datos o las pruebas automatizadas del repositorio — nunca en suposiciones. Donde una característica no puede verificarse, se marca explícitamente como `No implementado`, `No encontrado`, `No verificable` o `Implementación parcial`.

2. Objetivos

- Documentar el estado real (no el planificado) de los módulos funcionales de RentalBoosted.
- Evaluar el sistema contra las 8 características de calidad de ISO/IEC 25010, con evidencia objetiva.
- Servir de base para una futura auditoría de calidad de software.
- Identificar deuda técnica y hallazgos concretos, priorizados por impacto.

3. Alcance

Cubre: la aplicación Laravel del repositorio — sus 43 controladores, 45 modelos, 18 servicios, 39 migraciones, 235 rutas y 25 archivos de prueba —, su base de datos MySQL, y su integración con el microservicio externo API-GO (SUNAT).

No cubre: el código interno de API-GO (proyecto aparte, fuera de este repositorio), ni pruebas de carga o penetración reales — no existen en el proyecto; esa ausencia se documenta explícitamente en la sección correspondiente en vez de inventarse una métrica.

4. Descripción general del sistema

El sistema reemplaza una operación anterior basada en PHP procedural — el propio código documenta esto: `config/menu.php` indica en su encabezado que "reproduce exactamente las secciones, el orden, los iconos y los badges del sidebar de `administrador/index.php` del proyecto original" —, reconstruida por completo en Laravel.

Opera en 4 roles (`admin` , `secretaria` , `ventas` , `contador` , confirmados en `App\Models\Usuario::rutaInicio()`), cada uno con su propio panel y sidebar filtrado, y dos empresas activas: Casa Drywall (con SUNAT habilitado) y Jitk (sin SUNAT), aisladas por conexión de base de datos separada (`DB_JITK_*` en `.env` , middleware `SeleccionarEmpresa`).

5. Arquitectura

Arquitectura MVC estándar de Laravel, en capas: controladores delgados que delegan a Servicios (`app/Services/`) para lógica de negocio no trivial, modelos Eloquent como capa de persistencia, y vistas Blade + Vite para el frontend server-rendered (sin SPA ni API JSON propia para el navegador).

Flujo típico de una operación de negocio:

```
flowchart LR
    U[Usuario] --> F[Vista Blade]
    F -->|form POST| C[Controller]
    C --> M[Middleware auth + rol]
    M --> S[Servicio de negocio]
    S --> DB[(MySQL)]
    S -.opcional.-> API[API-GO / SUNAT]
    C --> R[Redirect o Vista]
```

Para multi-empresa, el middleware `SeleccionarEmpresa` decide, antes que cualquier controlador, qué conexión de base de datos usa el resto del request:

```
flowchart LR
    L[Login] --> SE[SeleccionarEmpresa]
    SE -->|Casa Drywall| DB1[(db_rentaltech)]
    SE -->|Jitk| DB2[(db_jitk)]
```

No se encontró una API REST propia orientada a consumidores externos (ver sección 14); la única integración saliente real es hacia API-GO.

6. Tecnologías utilizadas

Evidencia: `composer.json` y `package.json`.

Capa	Tecnología	Versión	Evidencia
Backend	PHP	^8.2	<code>composer.json</code>
Framework	Laravel	^12.0	<code>composer.json</code>
Base de datos	MySQL	driver <code>mysql</code>	<code>.env.example</code>
PDF	barryvdh/laravel-dompdf	^3.1	<code>composer.json</code>
Excel	phpoffice/phpspreadsheet	^5.9	<code>composer.json</code>
Frontend build	Vite	^7.0.7	<code>package.json</code>
CSS	Tailwind CSS	^4.0.0	<code>package.json</code>
HTTP cliente JS	Axios	^1.11.0	<code>package.json</code>
Gráficos/efectos	ogl	^1.0.11	<code>package.json</code>
Testing	PHPUnit	^11.5.50	<code>composer.json</code>
Estilo de código	Laravel Pint	^1.24	<code>composer.json</code>

`anthropic-ai/sdk` (^0.40.0) está declarado en `composer.json` y tiene una entrada en `config/services.php`, pero no se encontró ninguna llamada real a él en `app/` — dependencia sin uso verificado (ver Deuda Técnica, sección 20).

7. Módulos del sistema

43 controladores agrupados en 15 áreas funcionales. "Implementado" = tiene controlador, rutas, vista y al menos una prueba o flujo verificado en código; "Parcial" = existe pero con alcance reducido a propósito o sin pruebas.

Módulo	Funcionalidad	Estado	Evidencia	Observaciones
Ventas	Cotización / Nota de Venta / Boleta / Factura	Implementado	<code>VentaController.php</code> , <code>VentaMetodoPagoTest.php</code>	Un solo modelo <code>Venta</code> con <code>tipcomp</code> , no 4 entidades separadas
Ventas	Notas de Crédito / Débito	Implementado	<code>NotaCreditoDebitoTest.php</code>	Catálogo de motivos SUNAT 09/10
Ventas	Anulación / Eliminación (baja lógica)	Implementado	<code>VentasSubmenuTest.php</code>	Restringido si ya se envió a SUNAT

Módulo	Funcionalidad	Estado	Evidencia	Observaciones
Ventas	Generar venta desde Cotización	Implementado	VentaGenerarDesdeCotizacionTest.php	Enlace real por origen_cotizacion_id
POS	Punto de Venta con caja y pagos mixtos	Implementado	PosVentaTest.php , CajaTest.php	Bloquea venta si no alcanza el stock (a diferencia de Ventas normal)
Compras	Órdenes de Compra	Implementado	OrdenCompraWizardTest.php , OrdenCompraEditarTest.php	Editar productos ya existente; suma stock automático al Inventario
Compras	Proveedores	Implementado	ProveedorController.php	CRUD + catálogo de productos por proveedor
Compras	Cotización a Proveedor	Implementado	CotizacionProveedorController.php , ComprasTest.php	Correlativo propio, envío por correo
Compras	Activos Fijos	Implementado	ActivoFijoController.php , ComprasTest.php	Genera Egreso automático
Compras	Liquidación de Compra	Implementado (no SUNAT)	LiquidacionCompraController.php	Documento interno; el propio comprobante impreso aclara que no es válido ante SUNAT
Inventario	Productos / Catálogo	Implementado	ProductoController.php , ProductosTablaPreciosTest.php	Precios a 4 decimales
Inventario	Stock por almacén / Movimientos	Implementado	InventarioController.php , InventarioTest.php	Traslados, devoluciones, ajustes
Inventario	Kardex / Kardex Valorizado / Reporte	Implementado	CentroInventario.php	Costo ponderado = precio de compra actual (no promedio histórico real)
Inventario	Merch (catalogo promocional)	Implementado	MerchController.php , MerchInventario.php	Catálogo aparte de Productos
Finanzas	Cobranzas	Implementado	CobranzaController.php	Se genera automático al registrar una venta
Finanzas	Ingresos / Egresos	Implementado	IngresoController.php , EgresoController.php	—
Finanzas	Historial de Pagos	Implementado	HistorialPagoController.php	—
Finanzas	Catálogos (Bancos/Monedas/Tarjetas/Plataformas/Métodos de pago)	Implementado	BancoController.php + 4 controladores hermanos	CRUD idéntico repetido 5 veces (ver Deuda Técnica)
RR.HH.	Personal	Implementado	PersonalController.php , PersonalTest.php	Vincula opcionalmente un usuario de acceso
RR.HH.	Cargos	Implementado	CargoController.php	—
Logística	Transporte (tarifas)	Implementado	TransporteController.php	—
Logística	Guías de Remisión	Implementado	GuiaRemisionController.php	—
Reportes	ABC, Rotación, Aging CxC, Utilidad	Implementado	ReporteController.php , ReportesTest.php	Exporta a Excel y PDF

Módulo	Funcionalidad	Estado	Evidencia	Observaciones
Multi-empresa	Selector de empresa / Locales	Implementado	<code>EmpresaController.php</code> , <code>SeleccionarEmpresaTest.php</code>	Bases de datos separadas, no <code>empresa_id</code> compartido
Configuración	Mi Empresa / Estilos y Temas / Plantillas de Impresión	Implementado	<code>ConfiguracionController.php</code>	—
Configuración	Credenciales y certificados SUNAT	Solo lectura	<code>ApiGoCompanies.php</code>	Nunca editable desde RentalBoosted; valores sensibles ocultos
Notificaciones	Campanita (SUNAT pendiente, cobranza vencida, stock agotado)	Implementado	<code>CentroNotificaciones.php</code> , <code>CentroNotificacionesTest.php</code>	Calculadas en vivo, sin tabla de eventos propia
Pedidos	Pedidos de clientes	Implementado	<code>PedidoController.php</code> , <code>PedidoTest.php</code>	Con archivo adjunto opcional
Cotizaciones (legado)	Tablas <code>cotizaciones</code> / <code>cotizacion_detalle</code>	No utilizado	migración <code>2026_01_01_000300_create_comercial_tables.php</code>	Sin modelo Eloquent ni referencia en <code>app/</code> — código/esquema muerto (ver Deuda Técnica)

8. Requisitos funcionales

Requisitos verificables extraídos de los procesos de negocio principales (no exhaustivo para CRUD simples repetidos, ej. catálogos de Bancos/Monedas/Tarjetas, que siguen un mismo patrón estándar de alta/edición/baja).

RF-001 — Registrar comprobante de venta (Boleta/Factura) con envío a SUNAT

Descripción: Registrar una venta con Boleta o Factura, calculando IGV, y dejarla lista para enviarse a SUNAT vía API-GO. **Actor:** admin, ventas. **Precondiciones:** sesión activa; si hay productos del catálogo en el detalle, almacén elegido; medio de pago elegido. **Flujo principal:** 1. Completa cliente y productos o monto. 2. El sistema calcula base imponible/IGV/total. 3. Se crea la Cobranza y, desde ella, la Venta. 4. Se descuenta stock del almacén elegido. 5. Queda `estado_factura = pendiente`, lista para enviarse a SUNAT. **Resultado esperado:** Venta creada, Cobranza pendiente generada, stock actualizado. **Evidencia:** `app/Http/Controllers/Admin/VentaController.php::storeFactura()`; `tests/Feature/VentaFacturaStockTest.php`. **Estado:** Implementado.

RF-002 — Generar venta real desde una Cotización

Descripción: Convertir una Cotización (`tipcomp=COT`) en Nota de Venta, Boleta o Factura, precargando cliente y productos. **Actor:** admin, ventas. **Precondiciones:** Cotización existente y no eliminada. **Flujo principal:** 1. Desde el listado de Cotizaciones, elige el tipo a generar. 2. El formulario se precarga con cliente e ítems de la Cotización. 3. Al guardar, la nueva venta queda enlazada vía `origen_cotizacion_id`. **Resultado**

esperado: Nueva venta creada; la Cotización muestra "Convertida en" con el enlace real.

Evidencia: `VentaController.php::createFactura()/storeFactura()` ;
`tests/Feature/VentaGenerarDesdeCotizacionTest.php` . **Estado:** Implementado.

RF-003 — Anular o eliminar un comprobante (baja lógica)

Descripción: Marcar un comprobante como anulado o eliminado sin borrarlo de la base, revirtiendo el stock que hubiera descontado. **Actor:** admin, ventas (según el tipo).

Precondiciones: el comprobante no debe estar ya enviado a SUNAT (excepto Cotización/Nota de Venta, que nunca van a SUNAT). **Flujo principal:** 1. Se confirma la acción. 2. Si el comprobante ya salió a SUNAT, se rechaza con mensaje de usar Nota de Crédito. 3. Si es válido, se revierte el stock descontado y se marca `estado = cancelada` o `eliminada` . **Resultado esperado:** El comprobante desaparece del listado normal y aparece en "Anulaciones". **Evidencia:** `VentaController.php::anular()/destroy()` ;
`tests/Feature/VentasSubmenuTest.php` . **Estado:** Implementado.

RF-004 — Venta en Punto de Venta (POS) con bloqueo por stock

Descripción: Registrar una venta de mostrador con caja abierta, permitiendo pago mixto (varios métodos en una venta). **Actor:** admin, secretaria, ventas. **Precondiciones:** sesión de caja abierta. **Flujo principal:** 1. Se buscan productos y arman el carrito. 2. Se eligen uno o más métodos de pago. 3. El servidor recalcula precios/IGV (nunca confía en el total del navegador). 4. Si no alcanza el stock, la venta se rechaza (a diferencia de Ventas normal). **Resultado esperado:** Venta y pago(s) registrados, stock descontado con bloqueo de fila.

Evidencia: `app/Services/Pos/PosVentaService.php` ;
`tests/Feature/PosVentaTest.php` (incluye el caso de dos cajeros compitiendo por el último stock). **Estado:** Implementado.

RF-005 — Registrar Orden de Compra con actualización automática de Inventario

Descripción: Registrar una compra a proveedor cuyas líneas de producto sumen stock automáticamente al almacén elegido. **Actor:** admin. **Precondiciones:** ninguna obligatoria (el almacén destino es opcional; sin él, no se mueve stock). **Flujo principal:** 1. Se elige proveedor y productos. 2. Se elige almacén destino. 3. Al guardar, cada línea que coincide con un producto real del catálogo (por código) suma su cantidad al Inventario. **Resultado esperado:** Orden creada, stock del almacén elegido incrementado, movimiento de tipo `entrada` registrado. **Evidencia:** `app/Services/InventarioOrdenCompra.php` ;
`tests/Feature/OrdenCompraStockTest.php` . **Estado:** Implementado.

RF-006 — Editar una Orden de Compra ya registrada (cabecera y productos)

Descripción: Modificar los datos y las líneas de producto de una Orden de Compra existente, re-sincronizando el stock a la diferencia. **Actor:** admin. **Flujo principal:** 1. Se editan campos y/o líneas. 2. Al guardar, se revierte el stock que la orden había sumado antes y se reaplica según los valores nuevos. **Resultado esperado:** Orden actualizada; PDF y stock reflejan los datos nuevos sin duplicar ni perder movimientos. **Evidencia:** `OrdenCompraController.php::update()` ; `tests/Feature/OrdenCompraEditarTest.php`. **Estado:** Implementado.

RF-007 — Selección de empresa activa (multi-tenant)

Descripción: Antes de iniciar sesión, elegir entre las empresas configuradas (Casa Drywall, Jitk); el resto del sistema opera sobre la base de datos de la empresa elegida. **Actor:** cualquier visitante no autenticado. **Flujo principal:** 1. Se muestra el selector de empresas. 2. Al elegir una, `SeleccionarEmpresa` fija la conexión de base de datos en sesión. 3. Continúa al login. **Resultado esperado:** Todas las consultas del request usan la base de datos correcta; los datos de una empresa nunca aparecen en la otra. **Evidencia:** `app/Http/Middleware/SeleccionarEmpresa.php` ; `tests/Feature/SeleccionarEmpresaTest.php`. **Estado:** Implementado.

9. Requisitos no funcionales

ID	Requisito	Característica ISO 25010	Estado
RNF-001	Las escrituras concurrentes sobre stock y correlativos no deben corromper el dato	Fiabilidad	Implementado (<code>lockForUpdate()</code> + transacciones)
RNF-002	Una caída de API-GO/SUNAT no debe impedir registrar una venta	Fiabilidad	Implementado (prueba automatizada)

ID	Requisito	Característica ISO 25010	Estado
RNF-003	Cada request debe operar sobre la base de datos de la empresa activa, sin mezclar datos	Seguridad / Compatibilidad	Implementado
RNF-004	Acceso a cada módulo restringido por rol de usuario	Seguridad	Implementado (<code>VerificarRol</code> , 235 rutas)
RNF-005	Las contraseñas deben almacenarse con hash, nunca en texto plano	Seguridad	Implementado (<code>'password' => 'hashed'</code>)
RNF-006	Los formularios deben protegerse contra CSRF	Seguridad	Implementado (<code>@csrf</code> de Laravel)
RNF-007	El sistema debe responder en tiempos razonables bajo el uso normal de una PYME	Eficiencia de desempeño	No verificable — sin pruebas de carga ni métricas registradas
RNF-008	La configuración de producción debe evitar recompilar en cada request	Eficiencia de desempeño	No implementado — el despliegue nunca corre <code>config:cache</code> / <code>route:cache</code> / <code>view:cache</code>
RNF-009	El sistema debe poder instalarse en un entorno limpio siguiendo pasos documentados	Portabilidad	Implementado de forma parcial — pasos estándar de Laravel, sin guía propia del proyecto

ID	Requisito	Característica ISO 25010	Estado
RNF-010	El código debe tener pruebas automatizadas que cubran las reglas de negocio críticas	Mantenibilidad (capacidad de prueba)	Implementado — 25 archivos de prueba, cobertura no medida formalmente

10. Usuarios, roles y permisos

No hay Políticas ni Gates de Laravel (`app/Polices` vacío): la autorización completa del sistema pasa por un solo middleware, `App\Http\Middleware\VerificarRol`, aplicado por grupo de rutas en `routes/web.php` con la sintaxis `rol:admin`, `rol:admin,ventas`, etc. Un usuario sin el rol requerido es redirigido a su propio panel (`Usuario::rutaInicio()`), no a un error 403.

4 roles reales (columna `usuarios.rol`, valores confirmados en `Usuario::rutaInicio()`): `admin`, `secretaria`, `ventas`, `contador`.

Distribución real de las 235 rutas del sistema, por combinación de rol exigida (`php artisan route:list --json`):

Combinación de rol	Nº de rutas	Área típica
<code>admin</code> (solo)	145	Todo el panel administrativo: Compras, Inventario, Finanzas, RR.HH., Configuración, Reportes
<code>admin, ventas</code>	61	Módulo Ventas completo (Cotización/NV/Boleta/Factura, Notas de Crédito/Débito)
<code>admin, secretaria, ventas</code>	10	Punto de Venta (operación)
<code>admin, secretaria</code>	1	Catálogo de Cajas físicas
<code>secretaria</code> (solo)	1	Panel propio de Secretaria
<code>ventas</code> (solo)	1	Panel propio de Ventas
<code>contador</code> (solo)	1	Panel propio de Contador

Lectura: el rol `admin` puede llegar a todo; `secretaria` y `contador` tienen paneles muy acotados (su propio inicio, más POS para `secretaria`); `ventas` es el segundo rol más amplio, comparte casi todo el módulo comercial con `admin` pero no Compras, Inventario ni Finanzas.

No se encontró un sistema de permisos granular por acción (ej. "puede editar pero no eliminar"): el control es siempre a nivel de ruta/módulo completo.

11. Casos de uso

CU-01 — Emitir Boleta/Factura ante SUNAT

Actor: admin, ventas. **Objetivo:** dejar una venta lista y enviarla a SUNAT. **Precondiciones:** venta registrada con `estado_factura = pendiente`. **Flujo principal:** registrar venta (RF-001) → revisar comprobante → enviar a SUNAT desde el listado → API-GO responde aceptado/rechazado. **Flujos alternativos:** API-GO caído → la venta queda registrada igual, el envío se reintenta después. **Excepciones:** rechazo de SUNAT → queda visible en "No enviados"/estado de error. **Resultado:** comprobante con validez legal ante SUNAT.

CU-02 — Vender en el mostrador (POS)

Actor: admin, secretaria, ventas. **Objetivo:** cobrar una venta de mostrador con caja abierta. **Precondiciones:** sesión de caja abierta. **Flujo principal:** abrir caja → buscar productos → elegir método(s) de pago → confirmar venta. **Excepciones:** stock insuficiente → venta rechazada (único canal que bloquea por stock). **Resultado:** venta y movimiento de caja registrados.

CU-03 — Registrar una compra a proveedor

Actor: admin. **Objetivo:** dejar constancia de una compra y sumar el stock recibido. **Flujo principal:** elegir proveedor → agregar productos → elegir almacén destino → guardar. **Flujo alternativo:** línea con código fuera del catálogo → se guarda igual, pero no mueve stock. **Resultado:** Orden de Compra creada, Inventario actualizado.

CU-04 — Anular o eliminar un comprobante

Actor: admin, ventas. **Objetivo:** revertir un comprobante mal registrado. **Flujo principal:** elegir Anular/Eliminar → confirmar → se revierte stock si aplica → el comprobante pasa a "Anulaciones". **Excepción:** comprobante ya enviado a SUNAT → rechazado, se exige Nota de Crédito.

```

flowchart TD
    A[Anular / Eliminar] --> B{Enviado a SUNAT?}
    B -->|No| C[Revertir stock si aplica]
    C --> D[estado = cancelada / eliminada]
    B -->|Si| E[Rechazado: usar Nota de Credito]

```

12. Procesos de negocio

Ciclo de vida de un comprobante de venta (evidencia: `Venta.estado`, `estado_factura`, `VentaController.php`):

```

stateDiagram-v2
    [*] --> Activa: se registra
    Activa --> Pendiente_SUNAT: Boleta/Factura
    Pendiente_SUNAT --> Aceptado: API-GO/SUNAT OK
    Pendiente_SUNAT --> Rechazado: SUNAT rechaza
    Activa --> Cancelada: Anular (no enviado)
    Activa --> Eliminada: Eliminar (no enviado)
    Aceptado --> Corregido: Nota de Credito/Debito

```

Proceso Compra → Inventario (evidencia: `InventarioOrdenCompra.php`):

```

flowchart LR
    OC[Orden de Compra guardada] --> Rev[Revertir movimientos previos de esta orden]
    Rev --> Match{Linea coincide con Producto del catalogo?}
    Match -->|Si| Sum[Sumar cantidad a stock_almacen]
    Match -->|No| Skip[No toca stock]
    Sum --> Mov[Crear MovimientoAlmacen tipo entrada]
    Mov --> Recalc[Producto.recalcularStock]

```

Este patrón de "revertir todo lo de esta orden y reaplicar" (en vez de calcular una diferencia campo por campo) es el mismo que usa `MerchInventario`, y se eligió a propósito para que editar una orden nunca deje stock fantasma ni lo duplique.

13. Base de datos

MySQL, ~52 tablas reales creadas a lo largo de 39 migraciones versionadas (evidencia: `database/migrations/`). Diccionario completo de las tablas centrales; el resto se resume por área al final de la sección.

usuarios

Campo	Tipo	Obligatorio	PK	FK	Descripción
id	unsigned int	Sí	Sí		—
username	string(100)	Sí			Único
password	string	Sí			Hash bcrypt (cast hashed)
email	string(150)	No			—
rol	string(30)	Sí			admin secretaria ventas contador (default admin)
foto	string	No			Agregada en migración posterior

productos

Campo	Tipo	Obligatorio	PK	FK	Descripción
id	unsigned int	Sí	Sí		—
codigo	string(50)	No			Indexado, no único a nivel de esquema
nombre	string(255)	Sí			—
categoria_id / marca_id / proveedor_id	unsigned int	No		Sí (sin constraint FK explícita)	—
precio_compra / precio_venta	decimal(12,4)	No (default 0)			Ampliado de (10,2) a (12,4) el 2026-09-16
precio_alquiler	decimal(10,2)	No			Sin uso real (campo retirado del formulario)
stock	integer	No (default 0)			Acumulado, recalculado desde stock_almacen
viscosidad	string(50)	No			Sin uso real (campo retirado del formulario)

stock_almacen

Campo	Tipo	Obligatorio	PK	FK	Descripción
id	unsigned int	Sí	Sí		—
producto_id	unsigned int	Sí		Sí → productos.id (cascadeOnDelete)	—

Campo	Tipo	Obligatorio	PK	FK	Descripción
almacen_id	unsigned int	Sí		Sí → almacenes.id (cascadeOnDelete)	—
stock	integer	No (default 0)			Único por (producto_id, almacen_id)

movimientos_almacen

Campo	Tipo	Obligatorio	PK	FK	Descripción
id	unsigned int	Sí	Sí		—
producto_id / almacen_id	unsigned int	Sí		Sin constraint FK explícita	—
tipo	string(20)	Sí			entrada salida ajuste traslado devolucion
cantidad, stock_anterior, stock_nuevo	integer	No			—
usuario_id	unsigned int	No			Sin constraint FK explícita — rastro de quién hizo el movimiento
orden_compra_id	unsigned int	No			Agregado 2026-09-16, enlaza el movimiento a la Orden de Compra que lo generó
estado	string	No			en_curso entregado (agregado 2026-09-07)

ventas (tabla central del módulo comercial)

Cerca de 45 columnas tras 6 migraciones de ampliación sucesivas (SUNAT, POS, Notas de Crédito/Débito, Cotización). Columnas clave:

Campo	Tipo	Descripción
id	unsigned int	PK
tipcomp	string(5)	COT NV 03 (Boleta) 01 (Factura) 07 / 08 (Nota Crédito/Débito)
n_seri, n_comp	string	Serie y número del comprobante
estado	string(30)	activa (default) cancelada eliminada

Campo	Tipo	Descripción
estado_factura	string	Seguimiento ante SUNAT vía API-GO
cobranza_id	unsigned int (FK 1:1)	Cada venta nace de una Cobranza
almacen_id	unsigned int, nullable	Si hay producto real en el detalle, obligatorio en la validación de negocio
venta_origen_id	unsigned int, nullable	Solo Nota de Crédito/Débito: comprobante que corrige
origen_cotizacion_id	unsigned int, nullable	Venta real generada desde esta Cotización — self-referencia distinta de <code>venta_origen_id</code>
cod_motivo	string(2)	Solo NC/ND, catálogo SUNAT 09/10

venta_detalle / ordenes_compra

`venta_detalle`: `venta_id` (FK lógica), `producto_id` (nullable — línea manual sin catálogo), `cantidad`, `precio_unitario`, `subtotal`, `descuento_pct`. `ordenes_compra`: `numero_orden`, `proveedor`, `almacen_id` (nullable, agregado 2026-09-16), `productos` (**JSON**, no tabla normalizada — cada línea es un objeto `{codigo, descripcion, cantidad, precio_unit_usd}`), `estado`, `total_soles`.

Diagrama entidad-relación (núcleo comercial e inventario)

```
erDiagram
    PRODUCTOS ||--o{ STOCK_ALMACEN : tiene
    ALMACENES ||--o{ STOCK_ALMACEN : contiene
    PRODUCTOS ||--o{ MOVIMIENTOS_ALMACEN : afecta
    ALMACENES ||--o{ MOVIMIENTOS_ALMACEN : registra
    ORDENES_COMPRA ||--o{ MOVIMIENTOS_ALMACEN : genera
    VENTAS ||--o{ VENTA_DETALLE : contiene
    PRODUCTOS ||--o{ VENTA_DETALLE : referencia
    VENTAS ||--|| COBRANZAS : origina
    VENTAS ||--o| VENTAS : "origen_cotizacion_id (Cotizacion -> Venta)"
    VENTAS ||--o| VENTAS : "venta_origen_id (NC/ND -> comprobante)"
```

Otras tablas, por área (evidencia: nombre de migración, sin diccionario columna por columna)

Área	Tablas
Maestros	clientes, proveedores, personal, categorias, marcas, empresas_transporte, tarifas_transporte, email_contactos, config_sistema
Finanzas	cobranzas, ingresos, egresos, metas_mensuales, costos_mensuales, cuentas_bancarias, bancos, monedas, tarjetas, plataformas, metodos_pago
Compras	orden_tokens, activos_fijos, cotizaciones_proveedor, liquidaciones_compra
Logística	guias_remision
POS	cajas, sesiones_caja, movimientos_caja, venta_pagos, ventas_suspendidas
Merch	merch, merch_movimientos
RR.HH.	cargos
Sistema	notificacion_lecturas, estilos_sistema, perfil_negocio, plantilla_impresion, sessions, cache, jobs
Pedidos	pedidos_clientes

Hallazgo: esquema muerto

Las tablas `cotizaciones` y `cotizacion_detalle` (migración `2026_01_01_000300_create_comercial_tables.php`) **no tienen modelo Eloquent ni ninguna referencia en app/** — confirmado por búsqueda exhaustiva. La funcionalidad real de Cotización vive en `ventas` con `tipcomp = 'COT'`. Ver Deuda Técnica (sección 20).

14. API

No implementado un API REST propia orientada a consumidores externos. No existe `routes/api.php` cargado con endpoints de negocio, ni controladores que devuelvan JSON como contrato público versionado.

Lo que sí existe, y no debe confundirse con una API pública:

- **Endpoints AJAX internos**, consumidos solo por el propio frontend Blade/JS del sistema (ej. `admin/productos/buscar`, que devuelve JSON para los buscadores de producto de Ventas/Compras/Inventario). No tienen versionado ni documentación de contrato, y comparten la misma autenticación de sesión web que el resto del panel.
- **Consumo de una API externa:** RentalBoosted es *cliente* de API-GO (microservicio Laravel aparte, fuera de este repositorio) para la emisión electrónica SUNAT, vía `app/Services/Sunat/ApiGoEmissionService.php` y `app/Services/ApiGoSucursales.php / ApiGoCompanies.php`. Esa integración se documenta en Compatibilidad (sección 17.3) y Procesos de negocio, no aquí.

15. Seguridad

Evaluación de lo realmente implementado, no de lo que Laravel ofrece por defecto.

Aspecto	Estado	Evidencia
Autenticación	Implementado	Sesión web estándar de Laravel (<code>Auth::user()</code>), <code>usuarios.password</code> con cast <code>hashed</code> (<code>bcrypt</code>)
Autorización	Implementado (a nivel de ruta)	<code>VerificarRol</code> middleware, sin <code>Policies/Gates</code>
Gestión de sesiones	Implementado	Driver <code>database</code> (tabla <code>sessions</code>), <code>SESSION_LIFETIME=120</code> (<code>.env.example</code>)
CSRF	Implementado	<code>@csrf</code> en los formularios revisados de Ventas/Compras/Inventario
Validación de entradas	Implementado, parcial	<code>\$request->validate()</code> inline en controladores; 0 clases <code>FormRequest</code> encontradas (<code>app/Http/Requests</code> no existe)
SQL Injection	Mitigado por el ORM	Eloquent/Query Builder parametrizado en el código revisado; no se auditó línea por línea cada <code>whereRaw()</code>
XSS	Mitigado por Blade	<code>{{ }}</code> de Blade escapa por defecto; no se auditó cada uso de <code>{!! !!}</code>

Aspecto	Estado	Evidencia
Manejo de archivos	Implementado	Subida de logos/adjuntos vía <code>Storage::disk('public');</code> en Hostinger, <code>storage:link</code> no funciona (symlink/exec deshabilitados) y se resuelve a mano por SSH
Gestión de secretos	Implementado, delegado	Credenciales SOL/certificados SUNAT no se guardan en RentalBoosted; viven en API-GO. En Configuración solo se muestra "Configurada/No configurada", nunca el valor
Variables de entorno	Implementado	<code>.env</code> por instalación; hallazgo: <code>.env.example</code> trae <code>APP_DEBUG=true</code> — no se verificó el <code>.env</code> real de producción
Logs	Parcial	Canal <code>stack / single</code> de Laravel (errores de aplicación); sin log de auditoría de acciones de negocio (quien anuló qué, quien cambió qué configuración)
Autenticidad de comprobantes SUNAT	Delegado a API-GO	Certificado digital real lo maneja API-GO, no este repositorio

No se afirma que el sistema sea seguro por usar Laravel — los puntos sin evidencia directa (auditoría de `whereRaw / {!! !!}` uno por uno, prueba de penetración) se marcan como no verificados en este documento, no como "seguros por defecto".

16. Sistema de pruebas

25 archivos en `tests/Feature/` (PHPUnit 11, `RefreshDatabase`), únicamente pruebas de tipo Feature (HTTP + base de datos). **No se encontraron** pruebas Unit, de API, ni E2E (no hay Playwright/Dusk/Cypress configurado). `database/factories/` está vacío: los datos de prueba se crean directo con `Model::create([...])` en cada test, no con Factories de Laravel.

Área cubierta	Archivo(s)
Ventas / SUNAT	VentaFacturaStockTest, VentaEditarFacturaTest, VentaGenerarDesdeCotizacionTest, VentaMetodoPagoTest, VentasPanelTotalesTest, VentasSubmenuTest, NotaCreditoDebitoTest
POS / Caja	PosVentaTest, PosPantallaTest, CajaTest
Compras / Órdenes de Compra	OrdenCompraWizardTest, OrdenCompraEditarTest, OrdenCompraStockTest, ComprasTest, ProveedorProductosCatalogoTest, ProveedorProductosImportTest
Inventario / Productos	InventarioTest, ProductosTablaPreciosTest
Multi-empresa	SeleccionarEmpresaTest, LocalesTest
Otros	PedidoTest, PersonalTest, ReportesTest, CentroNotificacionesTest
Plantilla por defecto de Laravel	ExampleTest (falla en este proyecto: espera 200 en / , pero / redirige a login — no actualizada tras agregar autenticación obligatoria)

Cobertura no determinada. No se encontró configuración de Xdebug/PCOV ni reporte de cobertura (`.coverage` , `coverage.xml`) en el repositorio, y no se ejecutó una corrida con cobertura como parte de esta documentación. Módulos sin ningún archivo de test identificado: Finanzas (Cobranzas/Ingresos/Egresos/Historial de Pagos), RR.HH. (Cargos), Configuración, Guías de Remisión, Transporte.

17. Evaluación ISO/IEC 25010

17.1 Adecuación funcional

Compleitud: 15 áreas funcionales implementadas de punta a punta (sección 7); una sola área (Cotizaciones legado) es esquema muerto sin funcionalidad real. **Corrección:** 25 archivos de prueba verifican reglas de negocio reales (RF-001 a RF-007). **Pertinencia:** no se encontraron módulos "de mentira" — las áreas sin lógica real detrás (ej. antes "Plantillas de Impresión") se marcaban explícitamente como pendientes en vez de simularse. **Deficiencia real:** las tablas `cotizaciones` / `cotizacion_detalle` sugieren una funcionalidad que en realidad vive en otro lado (`ventas`) — riesgo de confusión para quien lea solo el esquema.

17.2 Eficiencia de desempeño

Comportamiento temporal: el despliegue a producción nunca ejecuta

`config:cache / route:cache / view:cache` (confirmado en el flujo de despliegue real de este proyecto) — cada request recompila configuración y vistas.

`VentaController::index()` (el listado completo de Ventas) **no pagina:** trae todas las filas que calcen el filtro con `->get()`, sin `paginate()` ni `limit()`. `ProductoController` sí limita explícitamente a 500 filas (`TOPE_LISTADO`). **Consultas a BD / N+1:** hay eager loading real en varios listados (ej. `Producto::activos()` -

`>with(['categoria','marca','stockPorAlmacen'])`), pero no se auditó

sistemáticamente cada uno de los 43 controladores para detectar N+1 puntuales — No verificable de forma exhaustiva en el alcance de este documento. **Paginación:** 16 de 43 controladores usan `->paginate()` (Proveedores, Clientes, Personal, Cargos, Egresos, Ingresos, Cobranzas, Guías, Merch, Compras extra, Categorías, Marcas, Inventario); los listados de mayor volumen real (Ventas, Productos) no paginan, usan carga completa o un tope fijo. **Caching:** `CACHE_STORE=database` (`.env.example`) — sin Redis/Memcached activo pese a estar disponibles como dependencias de Laravel. **Procesamiento pesado:** generación de PDF (dompdf) síncrona dentro del request; sin colas reales en uso (`QUEUE_CONNECTION=database` configurado, pero `app/Jobs` está vacío — 0 Jobs implementados). No existen métricas suficientes para determinar el comportamiento bajo carga.

17.3 Compatibilidad

Coexistencia: multi-empresa real, bases de datos separadas por `SeleccionarEmpresa` (sección 10). **Interoperabilidad:** única integración externa real es API-GO (SUNAT); diseño defensivo confirmado — una venta se registra igual aunque API-GO esté caído (prueba automatizada). Sin formatos de intercambio adicionales (no se encontró exportación/importación JSON/XML genérica, sí Excel vía `phpoffice/phpspreadsheet` en varios reportes).

17.4 Usabilidad / capacidad de interacción

Consistencia: diálogos nativos del navegador (confirm, selects, calendario) reemplazados por componentes propios en un solo punto central (`app.js`). **Responsive:** 47 reglas

@media reales en 14 hojas de estilo distintas — no es un diseño únicamente de escritorio.

Prevención de errores: confirmaciones antes de anular/eliminar, validación de servidor que bloquea acciones inválidas (ej. eliminar un comprobante ya enviado a SUNAT) con mensaje específico. **Feedback:** mensajes flash

(`session('mensaje')` / `session('error')`) tras cada acción. **No verificable:**

accesibilidad formal (lectores de pantalla, atributos ARIA, contraste WCAG) — sin auditoría ni evidencia en el código revisado.

17.5 Fiabilidad

Manejo de excepciones / transacciones: patrón consistente `DB::transaction()` + `lockForUpdate()` en POS, Ventas, Órdenes de Compra e Inventario (sección 12).

Recuperación ante errores: `destroy()` de Ventas pasó de borrado físico a baja lógica tras un incidente real en producción (cobranza huérfana) — evidencia de madurez creciente, no solo teórica. **Tolerancia a fallos:** caída de API-GO no bloquea el registro de la venta; dos cajeros compitiendo por el último stock del POS resuelven sin condición de carrera (prueba automatizada). **Logs:** solo de errores de aplicación, sin log de auditoría de negocio dedicado. **Disponibilidad:** sin monitoreo de uptime documentado; depende del hosting compartido.

17.6 Seguridad

Ver detalle completo en la sección 15. Resumen: autenticación/autorización sólidas a nivel de rol y ruta; CSRF activo; sin Policies/Gates ni `FormRequest`; sin log de auditoría de acciones administrativas; `.env.example` trae `APP_DEBUG=true` por defecto (no verificado el `.env` real de producción).

17.7 Mantenibilidad

Modularidad: 18 clases de Servicio dedicadas separan la lógica de negocio de los controladores. **Duplicación de código real detectada:** 5 controladores casi idénticos para catálogos simples (`BancoController`, `MonedaController`, `TarjetaController`, `PlataformaController`, `MetodoPagoController`) — mismo patrón CRUD repetido en vez de un controlador genérico parametrizado. **Reusabilidad:** `InventarioOrdenCompra` calca explícitamente el patrón de `MerchInventario` en vez de reinventarlo. **Testabilidad:** 25 archivos de prueba, sin Factories (dato de prueba creado inline en cada test).

17.8 Portabilidad

Adaptabilidad: configuración por `.env` estándar de Laravel; multi-empresa ya prueba adaptabilidad a más de una base de datos. **Instalabilidad:** sin Docker (no se encontró `Dockerfile` ni `docker-compose.yml` en el repositorio) — instalación manual con XAMPP en desarrollo. **Reemplazabilidad:** migraciones versionadas y en su mayoría no destructivas; despliegue a producción 100% manual (`zip` + `SSH` + `migrate`), sin CI/CD.

18. Matriz de evaluación ISO/IEC 25010

Estado usa únicamente las 4 categorías pedidas (`Implementado` / `Implementación parcial` / `No implementado` / `No verificable`) — sin puntuaciones numéricas arbitrarias, por no existir una metodología de medición objetiva definida de antemano

para este proyecto.

Característica	Subcaracterística	Evidencia	Estado	Observaciones
Adecuación funcional	Compleitud	15 áreas funcionales (sección 7)	Implementado	1 área es esquema muerto
Adecuación funcional	Corrección	25 archivos de prueba	Implementado	Cobertura no medida formalmente
Adecuación funcional	Pertinencia	Ausencia de módulos "de mentira"	Implementado	—
Eficiencia de desempeño	Comportamiento temporal	Sin <code>config:cache</code> en despliegue; <code>VentaController::index()</code> sin <code>paginar</code>	Implementación parcial	Riesgo de degradación con el crecimiento de datos
Eficiencia de desempeño	Utilización de recursos	PDF síncrono; <code>QUEUE_CONNECTION</code> configurado sin Jobs reales	Implementación parcial	—
Eficiencia de desempeño	Capacidad	Sin pruebas de carga	No verificable	—
Compatibilidad	Coexistencia	Multi-empresa con BD separada	Implementado	—
Compatibilidad	Interoperabilidad	<code>ApiGoEmissionService</code> , tolerante a caídas	Implementado	—
Usabilidad	Reconocibilidad de la adecuación	Sidebar organizado por área de negocio	Implementado	—
Usabilidad	Aprendizaje	Sin manual de usuario en el repositorio	No encontrado	—
Usabilidad	Operabilidad	Atajos de teclado en POS, sidebar filtrado por rol	Implementado	—
Usabilidad	Protección contra errores de usuario	Confirmaciones propias, validación de servidor con mensajes específicos	Implementado	—
Usabilidad	Estética de la interfaz	Tema oscuro consistente, 47 reglas responsive	Implementado	—
Usabilidad	Accesibilidad	Sin auditoría ARIA/contraste encontrada	No verificable	—
Fiabilidad	Madurez	Patrón transacción+lock repetido; incidentes reales corregidos	Implementado	—
Fiabilidad	Disponibilidad	Sin monitoreo de uptime	No verificable	Depende del hosting compartido
Fiabilidad	Tolerancia a fallos	Venta se registra con API-GO caído; concurrencia de stock probada	Implementado	—
Fiabilidad	Capacidad de recuperación	Reversión de stock en anular/eliminar, sin doble reversión	Implementado	—
Seguridad	Confidencialidad	Roles + sesión; secretos SUNAT delegados a API-GO	Implementado	—
Seguridad	Integridad	CSRF; transacciones con bloqueo	Implementado	—
Seguridad	No repudio	<code>usuario_id</code> en movimientos y ventas	Implementación parcial	Sin log de auditoría centralizado

Característica	Subcaracterística	Evidencia	Estado	Observaciones
Seguridad	Responsabilidad (accountability)	Rastro de quién registró cada operación de stock/venta	Implementación parcial	No cubre cambios de configuración
Seguridad	Autenticidad	Certificado digital real, gestionado por API-GO	Implementado (delegado)	Fuera de este repositorio
Mantenibilidad	Modularidad	18 servicios dedicados	Implementado	—
Mantenibilidad	Reusabilidad	InventarioOrdenCompra calca MerchInventario	Implementado	—
Mantenibilidad	Analizabilidad	Comentarios que explican el porqué, no el qué	Implementado	—
Mantenibilidad	Modificabilidad	Cambios recientes acotados sin tocar código no relacionado	Implementado	—
Mantenibilidad	Testabilidad	25 archivos de prueba, sin Factories	Implementación parcial	Varios módulos sin ningún test (sección 16)
Portabilidad	Adaptabilidad	Config por .env , multi-empresa ya probado	Implementado	—
Portabilidad	Instalabilidad	Sin Docker; storage:link no funciona en Hostinger (symlink/exec deshabilitados)	Implementación parcial	Se resuelve a mano por SSH
Portabilidad	Reemplazabilidad	Migraciones versionadas, mayormente no destructivas	Implementado	Sin rollback automatizado más allá de migrate:rollback

19. Hallazgos

HALL-001

Problema: El despliegue a producción nunca ejecuta

`config:cache / route:cache / view:cache`. **Evidencia:** flujo de despliegue real del proyecto (solo `config:clear / view:clear / cache:clear` en cada subida). **Impacto:** cada request recompila configuración y vistas en vez de servir versiones precompiladas, en un hosting ya de por sí compartido. **Característica ISO/IEC 25010:** Eficiencia de desempeño. **Recomendación:** agregar los 3 comandos `:cache` al final de cada despliegue. **Prioridad:** Alta.

HALL-002

Problema: Las tablas `cotizaciones / cotizacion_detalle` existen en el esquema pero no tienen modelo ni uso real — la funcionalidad de Cotización vive en `ventas`. **Evidencia:** migración `2026_01_01_000300_create_comercial_tables.php`; búsqueda sin resultados en `app/`. **Impacto:** confusión para cualquiera que documente o audite el esquema sin

conocer el historial del proyecto. **Característica ISO/IEC 25010:** Adecuación funcional (pertinencia) / Mantenibilidad (analizabilidad). **Recomendación:** eliminar las tablas en una migración de limpieza, o documentar explícitamente que son legado. **Prioridad:** Media.

HALL-003

Problema: `VentaController::index()`, el listado completo de Ventas, no pagina — trae todas las filas que calcen el filtro. **Evidencia:** `app/Http/Controllers/Admin/VentaController.php::index()`, sin `paginate()` ni `limit()`. **Impacto:** el tiempo de respuesta del listado de Ventas crecerá linealmente con el historial de la empresa, sin techo. **Característica ISO/IEC 25010:** Eficiencia de desempeño. **Recomendación:** paginar o acotar por rango de fechas por defecto (ya existe el filtro, solo falta forzarlo o paginar el resultado). **Prioridad:** Alta.

HALL-004

Problema: No hay log de auditoría para acciones administrativas sensibles (cambios de configuración, creación/baja de usuarios). **Evidencia:** sin tabla ni modelo de auditoría encontrado fuera de `movimientos_almacen / usuario_id` en ventas (que solo cubren stock y ventas). **Impacto:** difícil reconstruir quién cambió qué configuración del sistema ante un incidente. **Característica ISO/IEC 25010:** Seguridad (responsabilidad, no repudio). **Recomendación:** tabla de auditoría genérica (actor, acción, entidad, antes/después, fecha) para las pantallas administrativas críticas. **Prioridad:** Media.

HALL-005

Problema: 5 controladores casi idénticos (`BancoController`, `MonedaController`, `TarjetaController`, `PlataformaController`, `MetodoPagoController`) repiten el mismo patrón CRUD de catálogo simple. **Evidencia:** los 5 archivos en `app/Http/Controllers/Admin/`. **Impacto:** un cambio de comportamiento común (ej. validación de nombre único) hay que replicarlo 5 veces. **Característica ISO/IEC 25010:** Mantenibilidad (reusabilidad). **Recomendación:** extraer un controlador/servicio genérico de "catálogo simple" parametrizado por modelo. **Prioridad:** Baja.

HALL-006

Problema: `anthropic-ai/sdk` está declarado como dependencia y configurado en `config/services.php`, sin ninguna llamada real encontrada en `app/`. **Evidencia:** `composer.json`, `config/services.php`, búsqueda sin resultados de uso. **Impacto:** peso de dependencia y superficie de configuración sin beneficio actual. **Característica ISO/IEC 25010:** Mantenibilidad. **Recomendación:** retirar la dependencia si no hay un plan concreto de uso, o documentar para qué se reservó. **Prioridad:** Baja.

HALL-007

Problema: Sin clases `FormRequest` (validación inline en cada controlador) ni Policies/Gates de Laravel (autorización solo por middleware de rol a nivel de ruta).

Evidencia: `app/Http/Requests` y `app/Policies` no existen. **Impacto:** reglas de validación y autorización dispersas dentro de cada controlador, más difíciles de reutilizar y de auditar en un solo lugar. **Característica ISO/IEC 25010:** Mantenibilidad / Seguridad.

Recomendación: migrar las validaciones más repetidas a `FormRequest`; evaluar Policies para autorización más fina que "todo o nada por rol". **Prioridad:** Media.

HALL-008

Problema: El despliegue a producción es 100% manual (zip de archivos cambiados, subida por SSH, `unzip` + `migrate` a mano), sin CI/CD ni entorno de staging. **Evidencia:** proceso de despliegue real seguido en este proyecto; sin `.github/workflows` ni pipeline equivalente en el repositorio. **Impacto:** cada despliegue depende de que una persona no olvide un paso (migración, `npm run build`, sincronizar `public/build`); sin ambiente intermedio para detectar errores antes de producción. **Característica ISO/IEC 25010:** Portabilidad. **Recomendación:** al menos un script que empaquete y valide el deploy; idealmente CI/CD con un paso de staging. **Prioridad:** Alta.

HALL-009

Problema: `.env.example` trae `APP_DEBUG=true` por defecto. **Evidencia:** `.env.example`, línea `APP_DEBUG=true`. **Impacto:** si una instalación nueva copia el ejemplo sin cambiarlo, producción quedaría con mensajes de error detallados expuestos. **No se verificó el .env real de producción** (fuera del alcance de este análisis de código). **Característica ISO/IEC 25010:** Seguridad. **Recomendación:** cambiar el ejemplo a `APP_DEBUG=false` y confirmar el valor real en el `.env` de Hostinger. **Prioridad:** Media.

HALL-010

Problema: Varios módulos completos no tienen ningún archivo de prueba: Finanzas (Cobranzas/Ingresos/Egresos/Historial de Pagos), RR.HH. (Cargos), Configuración, Guías de Remisión, Transporte. **Evidencia:** lista de 25 archivos en `tests/Feature/` (sección 16) vs. 43 controladores. **Impacto:** cambios en esos módulos no tienen red de seguridad automatizada. **Característica ISO/IEC 25010:** Mantenibilidad (testabilidad). **Recomendación:** priorizar pruebas para Finanzas primero, por ser el módulo que maneja dinero sin cobertura hoy. **Prioridad:** Media.

20. Deuda técnica

Hallazgo	Clasificación	Motivo
<code>VentaController.php</code> con 1,323 líneas	Medio	Un solo controlador maneja Cotización, Nota de Venta, Boleta, Factura, Notas de Crédito/Débito, anulación, eliminación, envío a SUNAT y comprobante imprimible — múltiples responsabilidades en una sola clase, aunque ya delega cálculos a <code>PrecioCalculador</code>
Sin <code>config:cache / route:cache / view:cache</code> en producción	Alto	Afecta el desempeño de todo el sistema en cada request, no un módulo puntual (HALL-001)
<code>VentaController::index()</code> sin paginar	Alto	Crece sin límite con el historial real del negocio (HALL-003)
Tablas <code>cotizaciones / cotizacion_detalle</code> sin uso	Medio	Esquema muerto que puede confundir a futuros mantenedores (HALL-002)
5 controladores CRUD casi idénticos (Banco/Moneda/Tarjeta/Plataforma/MetodoPago)	Bajo	Duplicación mecánica, bajo riesgo de bug pero costo de mantenimiento repetido (HALL-005)
<code>anthropic-ai/sdk</code> sin uso real	Bajo	Solo peso de dependencia, sin riesgo funcional (HALL-006)

Hallazgo	Clasificación	Motivo
Sin <code>FormRequest</code> ni Políticas/Gates	Medio	Validación y autorización dispersas, dificulta auditar reglas en un solo lugar (HALL-007)
Despliegue 100% manual, sin CI/CD	Alto	Cada subida a producción depende de que una persona no olvide un paso (HALL-008)
Módulos sin ninguna prueba (Finanzas, RR.HH., Configuración, Guías, Transporte)	Medio	Sin red de seguridad automatizada para esas áreas (HALL-010)
Sin log de auditoría de acciones administrativas	Medio	Dificulta la trazabilidad ante un incidente que no sea de stock/ventas (HALL-004)

No se identificó ningún hallazgo de clasificación **Crítico** (ej. una vulnerabilidad de inyección confirmada, o pérdida de datos activa) dentro del alcance revisado.

21. Recomendaciones

Ordenadas por prioridad (Alta primero), consolidando los hallazgos de la sección 19:

1. **(Alta)** Agregar `config:cache / route:cache / view:cache` al final de cada despliegue a producción — HALL-001.
2. **(Alta)** Pagar o acotar por defecto el listado de Ventas (`VentaController::index()`) — HALL-003.
3. **(Alta)** Automatizar el despliegue (script mínimo, idealmente CI/CD con staging) — HALL-008.
4. **(Media)** Agregar un log de auditoría para acciones administrativas sensibles — HALL-004.
5. **(Media)** Confirmar `APP_DEBUG=false` en el `.env` real de producción y corregir el valor por defecto de `.env.example` — HALL-009.
6. **(Media)** Migrar las validaciones más repetidas a `FormRequest`; evaluar Políticas para autorización más fina — HALL-007.

7. **(Media)** Priorizar pruebas para Finanzas (maneja dinero sin cobertura hoy), luego el resto de módulos sin test — HALL-010.
8. **(Media)** Eliminar o documentar explícitamente las tablas `cotizaciones / cotizacion_detalle` como legado — HALL-002.
9. **(Baja)** Extraer un controlador/servicio genérico para los catálogos simples duplicados — HALL-005.
10. **(Baja)** Retirar `anthropic-ai/sdk` si no hay plan de uso, o documentar su propósito — HALL-006.

22. Matriz de trazabilidad

Requisito	Funcionalidad	Código	BD	Prueba	ISO 25010
RF-001	Registrar Boleta/Factura con SUNAT	<code>VentaController::storeFactura()</code>	<code>ventas, venta_detalle, cobranzas</code>	<code>VentaFacturaStockTest.php</code>	Adecuación funcional, Compatibilidad
RF-002	Generar venta desde Cotización	<code>VentaController::createFactura()/storeFactura()</code>	<code>ventas.origen_cotizacion_id</code>	<code>VentaGenerarDesdeCotizacionTest.php</code>	Adecuación funcional
RF-003	Anular/Eliminar comprobante	<code>VentaController::anular()/destroy()</code>	<code>ventas.estado</code>	<code>VentasSubmenuTest.php</code>	Fiabilidad, Seguridad
RF-004	Venta en POS con bloqueo de stock	<code>PosVentaService.php</code>	<code>ventas, stock_almacen, sesiones_caja, venta_pagos</code>	<code>PosVentaTest.php</code>	Fiabilidad
RF-005	Orden de Compra actualiza Inventario	<code>InventarioOrdenCompra.php</code>	<code>ordenes_compra, stock_almacen, movimientos_almacen</code>	<code>OrdenCompraStockTest.php</code>	Adecuación funcional
RF-006	Editar Orden de Compra	<code>OrdenCompraController::update()</code>	<code>ordenes_compra</code>	<code>OrdenCompraEditarTest.php</code>	Mantenibilidad
RF-007	Selección de empresa activa	<code>SeleccionarEmpresa.php</code>	conexión de BD completa (no una tabla)	<code>SeleccionarEmpresaTest.php</code>	Compatibilidad, Seguridad

23. Instalación

Pasos reales, tomados de `README.md` del propio repositorio (verificados, no inventados).

Requisitos: PHP 8.2+, MySQL 8, Node 20+.

```
composer install
npm install
cp .env.example .env           # ajustar credenciales de base de datos
php artisan key:generate
php artisan migrate --seed
npm run build                   # o npm run dev en desarrollo
php artisan serve
```

Usuarios de ejemplo (creados por el seeder, según el README — cambiar antes de producción):

Usuario	Contraseña	Rol	Panel
admin	admin123	admin	/admin
secretaria	secretaria123	secretaria	/secretaria
contador	contador123	contador	/contador

El README no lista un usuario de ejemplo para el rol `ventas` (que sí existe en el sistema, sección 10) — posible desactualización del README, ver Anexos.

El registro público (`/register`) exige una **contraseña maestra** para poder crear usuarios nuevos (`RT_MASTER_PASSWORD` , default `654321`) — confirmado en `RegisterController.php` , no solo declarado en config.

24. Configuración

Variables de entorno reales, tomadas de `.env.example` :

Variable	Propósito
<code>APP_DEBUG</code>	<code>true</code> por defecto en el ejemplo (ver HALL-009)
<code>APP_LOCALE</code> / <code>APP_FALLBACK_LOCALE</code>	<code>es</code>
<code>DB_CONNECTION</code> , <code>DB_HOST</code> , <code>DB_DATABASE</code> , etc.	Conexión principal (Casa Drywall, <code>db_rentaltech</code>)
<code>DB_JITK_HOST</code> , <code>DB_JITK_DATABASE</code> , etc.	Conexión separada de la segunda empresa (Jitk)
<code>SESSION_DRIVER</code>	<code>database</code>
<code>CACHE_STORE</code>	<code>database</code> (sin Redis/Memcached activo)
<code>QUEUE_CONNECTION</code>	<code>database</code> (configurado, sin Jobs implementados)
<code>RT_MASTER_PASSWORD</code>	Contraseña exigida para autorregistrarse un usuario nuevo

Configuración de negocio en base de datos (no solo `.env`): Mi Empresa (`perfil_negocio`), Estilos y Temas (`estilos_sistema`), Plantillas de Impresión (`plantilla_impresion`), Cuentas Bancarias, y los catálogos de Bancos/Monedas/Tarjetas/Plataformas/Métodos de pago — todos editables desde el propio panel de Configuración, sin tocar archivos.

25. Despliegue

Servidor: Hostinger, hosting compartido. **Sistema operativo / runtime:** Linux con PHP gestionado por el panel de Hostinger (versión exacta no verificada en este análisis de código). **Base de datos:** MySQL remota del mismo panel.

Restricciones reales confirmadas del entorno: `symlink()` y `exec()` deshabilitados en PHP — `php artisan storage:link` falla ahí; el symlink de `storage` se crea a mano por SSH (`ln -s`). El sitio se sirve desde `public_html/`, una carpeta distinta de `public/` (la que genera Laravel) — los assets compilados (`public/build/`) deben copiarse a mano a `public_html/build/` tras cada `npm run build`.

Proceso de despliegue real (manual, sin pipeline):

1. Commit y push a `origin/master`.
2. Si cambió CSS/JS: `npm run build`.
3. Empaquetar en `.zip` solo los archivos de código modificados (más `public/build/` completo si hubo build).
4. Subir por SSH y `unzip -o` sobre el proyecto en producción.
5. `php artisan migrate --force` si hubo migraciones nuevas.
6. `php artisan config:clear && view:clear && cache:clear`.

Workers / cron jobs: no se encontró ningún Job ni Command propio programado (`app/Jobs` y `app/Console` sin comandos custom) — No implementado. **Servicios externos en producción:** API-GO (microservicio SUNAT, con certificado digital real y `modo_produccion` activo para Casa Drywall).

26. Conclusiones técnicas

RentalBoosted es un sistema funcionalmente completo y correctamente probado en sus flujos de negocio críticos (Ventas/SUNAT, POS, Compras-Inventario), con un patrón consistente de transacciones y bloqueo de fila que le da fiabilidad real, no solo teórica — confirmado por incidentes reales ya diagnosticados y corregidos (ej. la cobranza huérfana que llevó a convertir `destroy()` en baja lógica).

Las debilidades reales no están en la lógica de negocio sino alrededor de ella: el despliegue a producción es manual y no activa el cacheo de Laravel (Eficiencia de desempeño, Portabilidad), y varios módulos administrativos completos no tienen ninguna prueba automatizada (Mantenibilidad). Ninguno de los hallazgos de esta evaluación se clasificó como Crítico dentro del alcance revisado.

Este documento cubre el código, la configuración y las pruebas del repositorio; no reemplaza una auditoría de seguridad formal (pentesting) ni una prueba de carga real, ninguna de las cuales existe hoy para este proyecto.

27. Anexos

A. Glosario de estados reales

- `Venta.estado`: `activa` (default) | `cancelada` | `eliminada`.
- `Venta.estado_factura`: `pendiente` (default) | `registrado` | `aceptado` | `rechazado`.
- `Venta.tipcomp`: `COT` (Cotización) | `NV` (Nota de Venta) | `03` (Boleta) | `01` (Factura) | `07` (Nota de Crédito) | `08` (Nota de Débito).
- `MovimientoAlmacen.tipo`: `entrada` | `salida` | `ajuste` | `traslado` | `devolucion`.
- `MovimientoAlmacen.estado`: `en_curso` | `entregado`.

B. Discrepancia detectada en la documentación existente

El `README.md` del repositorio describe módulos que ya no corresponden al estado actual del código — ej. menciona "Cotizaciones" como área separada de "Ventas" (hoy es un `tipcomp` dentro de `ventas`, sección 7), "Facturas (+ análisis de PDF con IA)" (sin implementación de IA encontrada; posible origen histórico de la dependencia `anthropic-ai/sdk` sin uso, HALL-006), y "Galonaje" dentro de Reportes/Análisis (esas columnas se eliminaron explícitamente en la migración `2026_09_11_094307_drop_galonaje_columns.php`). El README documenta una versión anterior del sistema, no la actual.

C. Inventario de controladores (evidencia sección 7)

43 controladores en `app/Http/Controllers/` (39 en `Admin/`, 2 en `Auth/`, más `ContadorController`, `EmpresaController`, `PerfilController`, `SecretariaController`, `VentasController` — estos 5 últimos son los controladores de los paneles propios de cada rol no-admin y del selector de empresa).

D. Metodología de esta documentación

Generada mediante inspección directa del código (controladores, modelos, servicios, middleware, migraciones, rutas vía `php artisan route:list`, archivos de prueba) y de la configuración (`composer.json`, `package.json`, `.env.example`, `README.md`) del repositorio en su estado al commit `b232068` (18 de septiembre de 2026). No se ejecutaron pruebas de carga, penetración, ni auditoría de accesibilidad como parte de este trabajo — sus secciones correspondientes quedan marcadas `No verificable` a propósito.